

第 28 讲

PyTorch 中的 Linear 与 Activation

nn.Linear 到底替我们做了什么？

《面向 LLM / Agent Systems 科研的深度学习零基础课》

【本讲定位】第 27 讲已经把 NumPy ndarray 过渡到 torch.Tensor。现在把前面手写过很多次的 $Y = XW + b$ 、ReLU、GELU 等运算，逐项对照到 PyTorch 的 nn.Linear 与 Activation 模块。目标不是“学会调用 API”，而是看到一行 linear(x) 时，脑中能还原出 Weight、Bias、Shape 与矩阵乘法。

学习目标

学完本讲，你应该能够：

1. 解释 nn.Linear(in_features, out_features) 内部保存了什么；
2. 读懂 weight.shape=(out,in) 与 bias.shape=(out,);
3. 解释为什么 PyTorch 的前向可写成 $XW^T + b$ ；
4. 用已知 Weight/Bias 手算一次 nn.Linear；
5. 计算 Linear Layer 的参数数量；
6. 理解 nn.Linear 对二维 Batch 和三维 (B, T, D) Tensor 的 Shape 规则；
7. 区分 Linear 与 Activation 的职责；
8. 比较 ReLU、GELU、SiLU 在 PyTorch 中的输入输出；
9. 理解 nn.Sequential 与 state_dict 的最小用途；
10. 把这些概念重新连接到 Transformer FFN / MLP。

1 【原理】nn.Linear 本质上仍然是我们学过的 affine mapping

第 5、7、10、25 讲已经反复出现：

$$Y = XW + b$$

本质上就是：

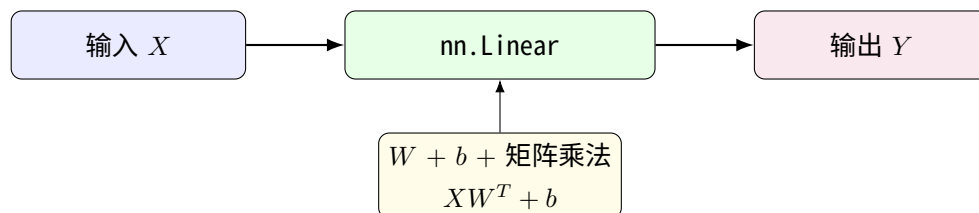
1. 输入与 Weight 做加权组合；

2. 再加 Bias;
3. 得到新的特征表示。

PyTorch 没有发明新的数学。nn.Linear 做的是同一类计算，只是把：

- Weight 的创建与保存;
- Bias 的创建与保存;
- Shape 检查;
- Batch / 多维 Tensor 的批量计算;
- 与 Autograd 的连接;

统一封装成一个 Layer。



你以后看到 nn.Linear，不要只读成“一个 PyTorch 层”。要在脑中自动还原为：一组可训练 Weight + 一组可训练 Bias + 一次 affine transformation。

2 【通俗解释】为什么 PyTorch 的 Weight Shape 看起来“反了”？

这一点非常容易混淆。

如果从批量行向量的角度，我们前面常写：

$$X : (B, D_{in}), \quad W : (D_{in}, D_{out})$$

于是：

$$XW : (B, D_{out})$$

但 PyTorch 的：

```
layer = nn.Linear(in_features=3, out_features=2)
```

内部保存：

$$weight.shape = (2, 3)$$

也就是：

$$(D_{out}, D_{in})$$

因此批量前向等价于：

$$Y = XW^T + b$$

这里的 W 指 PyTorch 实际保存的 weight。

这不是数学矛盾，只是存储约定不同。单样本如果把输入写成列向量，教材常写 $y = Wx + b$ ；批量样本在代码中常按“每个样本占一行”存放，所以对应写成 $XW^T + b$ 。

3 【具体数字例子】手算一个 `nn.Linear(3,2)`

设输入：

$$x = [1, 2, -1]$$

PyTorch 内部 Weight：

$$W = \begin{bmatrix} 0.5 & -1 & 2 \\ 1 & 0.5 & -0.5 \end{bmatrix}$$

Bias：

$$b = [0.2, -0.3]$$

因为每一行 Weight 对应一个输出特征，所以第一个输出：

$$\begin{aligned} y_1 &= 1 \times 0.5 + 2 \times (-1) + (-1) \times 2 + 0.2 \\ &= 0.5 - 2 - 2 + 0.2 = -3.3 \end{aligned}$$

第二个输出：

$$\begin{aligned} y_2 &= 1 \times 1 + 2 \times 0.5 + (-1) \times (-0.5) - 0.3 \\ &= 1 + 1 + 0.5 - 0.3 = 2.2 \end{aligned}$$

所以：

$$y = [-3.3, 2.2]$$

写成批量/行向量形式：

$$y = xW^T + b$$

一个 `nn.Linear(3,2)` 可以理解成“两个神经元并排”：每个神经元都接收 3 个输入，各自拥有 3 个 Weight 和 1 个 Bias，因此一共 $(3 \times 2) + 2 = 8$ 个可训练参数。

4 【参数量】Linear Layer 里到底有多少参数？

一般地：

$$\text{nn.Linear}(D_{in}, D_{out})$$

有：

$$D_{in} \times D_{out}$$

个 Weight，加上：

$$D_{out}$$

个 Bias。

因此：

$$\#Params = D_{in}D_{out} + D_{out}$$

例如：

$$\text{nn.Linear}(768, 3072)$$

参数量为：

$$768 \times 3072 + 3072 = 2,362,368$$

约 236 万个参数。

【科研阅读】论文里的 hidden size、FFN expansion ratio、projection dimension 不只是“Shape 设置”，它们会直接改变参数量、显存、计算量和表示容量。以后看到结构消融时，应主动计算这些改变是否同时引入了参数量差异。

5 【代码实验】怎样把已知 Weight / Bias 填进 nn.Linear?

```
linear = nn.Linear(3, 2)

with torch.no_grad():
    linear.weight.copy_(known_weight)
    linear.bias.copy_(known_bias)

x = torch.tensor([1.0, 2.0, -1.0])
y = linear(x)
```

为什么使用：

```
with torch.no_grad():
```

因为 `linear.weight` 和 `linear.bias` 默认都是需要梯度的 `Parameter`。这里我们只是为了实验，把它们人为替换成已知值，不希望这个“赋值动作”进入自动微分记录。

正式理解 Autograd 放在第 29 讲。

6 【Shape】二维 Batch：Linear 改变最后一个特征维

如果：

$$X.shape = (B, D_{in})$$

调用：

$$\text{Linear}(D_{in}, D_{out})$$

得到：

$$Y.shape = (B, D_{out})$$

例如：

$$(32, 768) \rightarrow \text{Linear}(768, 3072) \rightarrow (32, 3072)$$

这意味着同一个 Linear Layer 会对 Batch 中的 32 个样本共享同一套 Weight 与 Bias。

7 【LLM 连接】三维 (B, T, D) Tensor 为什么可以直接送进 Linear?

Transformer 中更常见的是：

$$X.shape = (B, T, D)$$

其中：

- B ：Batch Size；
- T ：Sequence Length / Token 数；
- D ：每个 Token 的 hidden dimension。

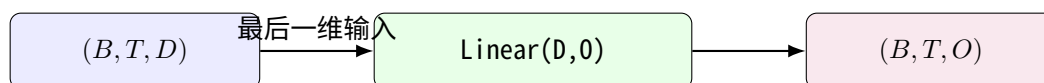
如果：

$$\text{Linear}(D, O)$$

那么：

$$(B, T, D) \rightarrow (B, T, O)$$

前面的 B, T 保持不变，Linear 只映射最后一维。



每个 Token 独立使用
同一套 Weight/Bias

普通 Linear 自己并不会让不同 Token 互相交换信息。它主要是对每个 Token 的特征向量做同一套映射。Transformer 中 Token 之间真正建立数据依赖的是 Attention。

8 【原理】Activation 又在做什么？

Linear 负责：

重组 / 混合特征

Activation 负责：

对这些数值做非线性映射

典型结构：

$$X \rightarrow \text{Linear} \rightarrow \text{Activation} \rightarrow \text{Linear}$$

PyTorch 中可以写：

```

relu = nn.ReLU()
gelu = nn.GELU()
silu = nn.SiLU()
  
```

如果：

$$z = [-3, -1, 0, 1, 3]$$

那么 ReLU：

$$\text{ReLU}(z) = [0, 0, 0, 1, 3]$$

GELU 和 SiLU 更平滑，会保留部分负值。它们在第 9 讲已经从底层学过，现在只是把公式映射到 PyTorch Module。

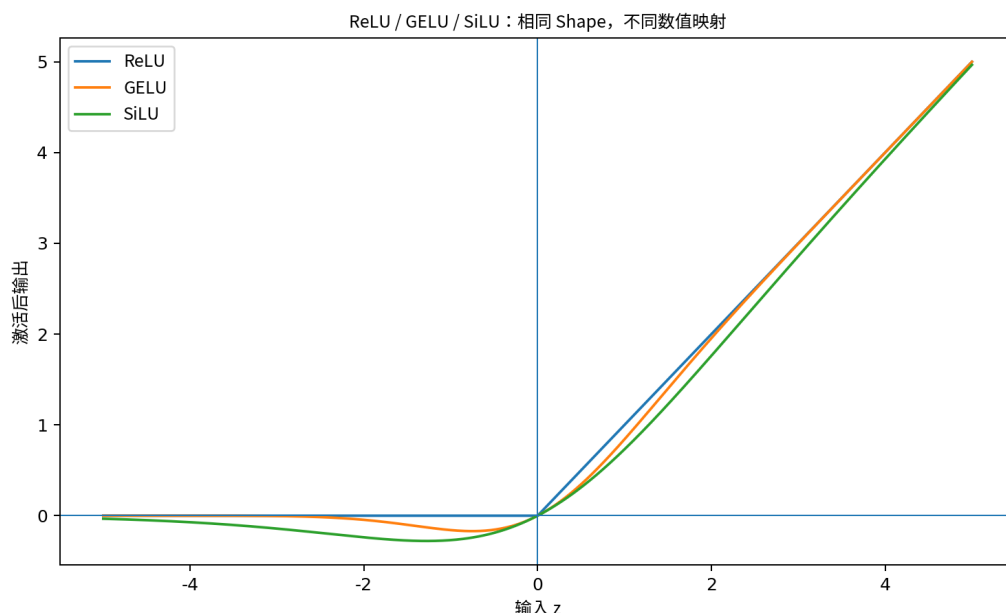


图 1: 实验：ReLU、GELU、SiLU 在相同输入下的不同数值映射。

9 【重要 Shape 直觉】Activation 通常改变数值，不改变 Shape

对于逐元素激活：

$$(B, T, D) \xrightarrow{\text{GELU}} (B, T, D)$$

例如：

$$(2, 3, 8) \xrightarrow{\text{GELU}} (2, 3, 8)$$

Shape 没有变，但每一个位置的数值都按照 GELU 规则重新映射。

这和 Linear 很不同：

$$(2, 3, 4) \xrightarrow{\text{Linear}(4,8)} (2, 3, 8)$$

Linear 可以改变最后一个维度。

组件	主要作用	Shape 常见变化
Linear	混合/重组特征	最后一维 $D_{in} \rightarrow D_{out}$
ReLU/GELU/SiLU	加入非线性	通常 Shape 不变

10 【Module 与 Functional】nn.ReLU() 和 F.relu() 有什么区别？

下面两种写法可以得到相同数值：

```
relu = nn.ReLU()
y1 = relu(x)

y2 = torch.nn.functional.relu(x)
```

简单理解：

- nn.ReLU()：一个 Module 对象，适合放进模型结构、nn.Sequential；
- F.relu(x)：直接调用函数。

对于 ReLU 这种没有可训练参数的激活，两种写法都很常见。

不要因此推广成“所有 Module 都可以随便换成 functional”。有些模块包含可训练参数、buffer 或 train/eval 状态，需要更谨慎地区分。当前只建立最小直觉。

11 【nn.Sequential】把层按顺序装起来

上一阶段我们手写 MLP：

$$Linear_1 \rightarrow GELU \rightarrow Linear_2$$

PyTorch 可以直接写：

```
mlp = nn.Sequential(
    nn.Linear(3, 4),
    nn.GELU(),
    nn.Linear(4, 2),
)
```

参数量：

$$(3 \times 4 + 4) + (4 \times 2 + 2) = 26$$

注意 GELU 没有增加可训练 Weight/Bias，因此这里参数量全部来自两个 Linear Layer。

12 【state_dict】模型里真正需要保存的东西

对上面的 MLP：

```
for name, tensor in mlp.state_dict().items():
    print(name, tensor.shape)
```

会看到类似：

```
0.weight (4, 3)
0.bias (4,)
2.weight (2, 4)
2.bias (2,)
```

这帮助我们建立一个重要直觉：

“模型”并不是一个神秘文件。它至少包含一套架构定义，以及这套架构中需要保存的参数 Tensor。

以后做科研实验时，checkpoint、state_dict、随机初始化、预训练权重是否一致，会直接影响可复现性。不能只记录“用了同一个模型名字”，还要明确权重来源和版本。

13 【与 Transformer / LLM 的关系】FFN 本质上就是今天的组件

一个极简 Transformer FFN 可以写成：

$$(B, T, D) \xrightarrow{\text{Linear}(D, H)} (B, T, H) \xrightarrow{\text{GELU}} (B, T, H) \xrightarrow{\text{Linear}(H, D)} (B, T, D)$$

例如小实验：

$$(2, 3, 4) \rightarrow (2, 3, 8) \rightarrow (2, 3, 8) \rightarrow (2, 3, 4)$$

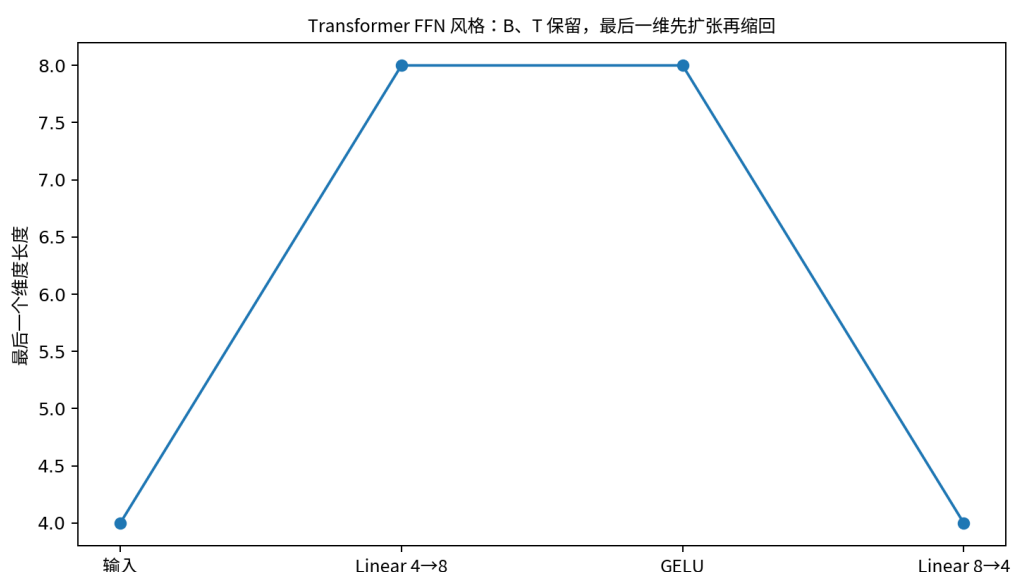


图 2: 实验：Transformer FFN 风格结构中，最后一维先扩张再缩回。

你以前在 LLM 课程里见过 FFN / MLP。现在要把它重新理解成：

1. 第一个 Linear：每个 Token 的特征从 D 映射到更大的隐藏维 H ；
2. Activation：对每个数值施加非线性；
3. 第二个 Linear：再从 H 映射回 D ；
4. B 和 T 通常在这一段保持不变。

14 【常见误区】

1. **误区：**`nn.Linear` 是一种新的数学。
纠正：它仍然是 affine transformation，只是框架封装了参数与批量计算。
2. **误区：**PyTorch 的 Weight Shape 与前面公式冲突。
纠正：只是存储约定不同；批量前向是 $XW^T + b$ 。
3. **误区：**Activation 会改变 Tensor 的维度。
纠正：ReLU/GELU/SiLU 这类逐元素激活通常保持 Shape。
4. **误区：**ReLU 也有大量可训练参数。
纠正：普通 ReLU 没有 Weight/Bias；参数主要来自 Linear 等层。
5. **误区：**`nn.Linear` 对三维 Tensor 会把所有 Token 混到一起。
纠正：它主要对最后一个维度应用同一套映射；不同 Token 的直接交互不是由普通 Linear 完成。
6. **误区：**看到 `requires_grad=True` 就已经有梯度了。
纠正：这里只说明参数参与自动微分；正式梯度要等第 29 讲调用 `backward`。

15 【科研阅读】看到一个 Layer 时，先问哪五件事？

以后看论文结构图或 PyTorch 代码，建议先问：

1. 输入 Shape 是什么？
2. 输出 Shape 是什么？
3. 这个 Layer 有没有可训练参数？有多少？
4. 它改变的是“维度”，还是只改变“数值”？
5. 它有没有让不同 Token / 样本之间发生信息交换？

这五个问题能把大量看起来复杂的架构拆回底层。

本讲小结

`nn.Linear` = 可训练 Weight + Bias + affine transformation；Activation = 非线性数值映射。PyTorch 只是把我们已经手写过的底层计算封装成可组合、可训练的 Module。

现在看到：

```
x = linear(x)
x = gelu(x)
```

你应该能够还原：

$$X \xrightarrow{XW^T+b} Z \xrightarrow{GELU} A$$

并预测每一步的 Shape。

下一讲进入：**第 29 讲《Autograd：loss.backward() 到底怎样自动得到梯度》**。我们会把第 22-26 讲手算的 Backpropagation，与 PyTorch 自动生成的梯度逐项对照。